

DURANTA INSTALLATION GUIDE

INTRODUCTION –

Duranta is an open-source 5G radio access network codebase based on OpenAirInterface 5G. It allows us to build and run a local 5G network on a Linux computer without needing expensive physical hardware.

This guide explains how to:

- clone the Duranta repository
- install dependencies
- build the project
- run the 5G gNB and UE locally
- connect the gNB to the UE using RF simulation

The main goal is to understand how a local 5G network works internally.

HOW THE LOCAL 5G NETWORK WORKS?

A normal 5G network has three main parts:

- UE: mobile phone or device
- gNB: base station or tower
- 5G Core Network: handles internet and subscriber management

In local simulation, we can run the UE and gNB on the same computer. The RF simulator replaces real radio waves with software messages over localhost.

So instead of actual antennas:

- gNB sends simulated radio data
- UE receives it
- both communicate through software

This is useful for testing and learning.

Duranta End-to-End Workflow (gNB ↔ AMF ↔ UE)

1. Setup Phase

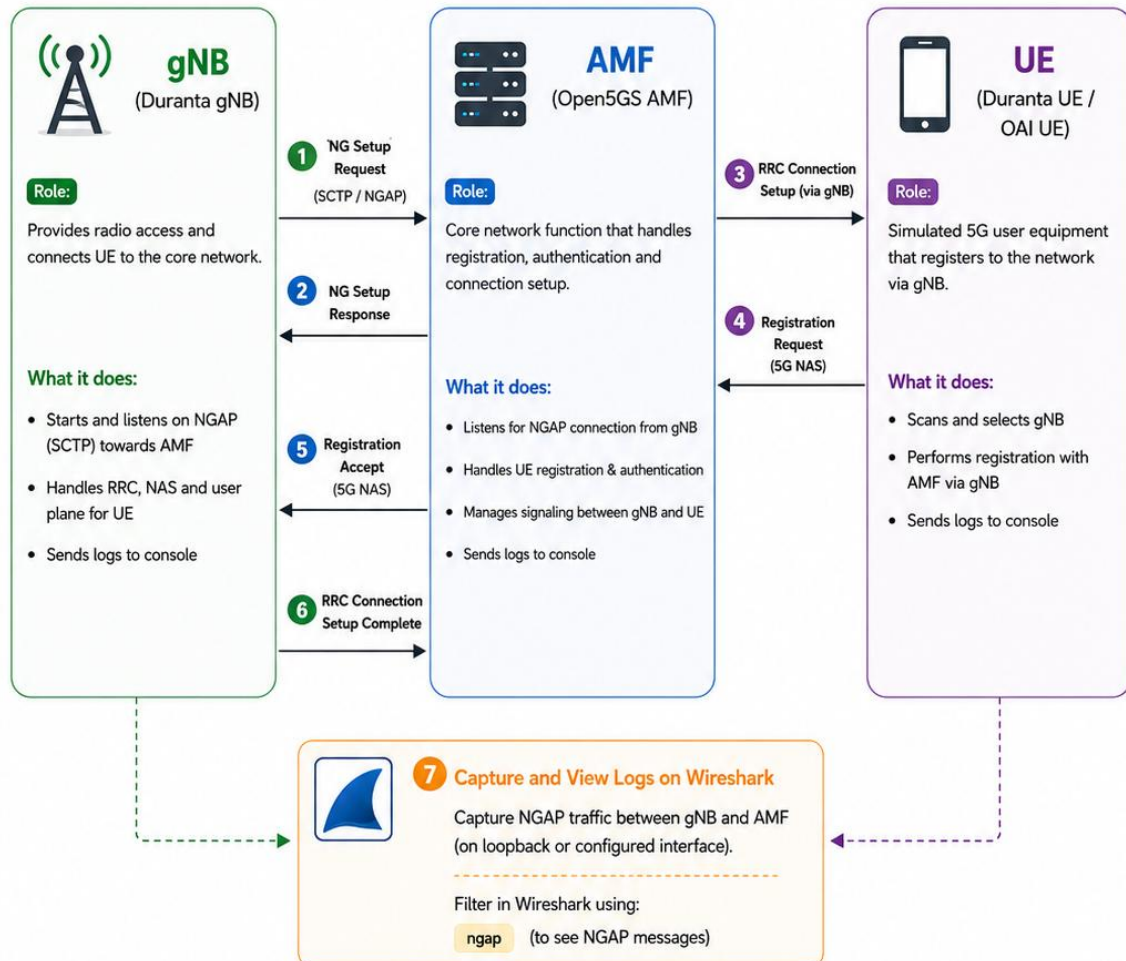
Before any communication happens, all three components must be running.

- Start the Open5GS AMF first. The AMF acts as the 5G Core Network and waits for

incoming NGAP connections from the gNB.

- Start the Duranta gNB. The gNB establishes a control-plane connection with the AMF.

Duranta – End to End Flow (gNB ↔ AMF ↔ UE)



Overall Setup Sequence



Ports Used (Default)

- NGAP (SCTP): 38412 (gNB ↔ AMF)
- S1AP (Not used in 5G SA)
- N2 over SCTP

Notes

- All components can run on the same machine (localhost setup).
- Ensure IP and port configurations match in gNB and AMF configs.
- Use Wireshark to observe NGAP messages and UE registration flow.

- Start the Duranta UE. The UE begins searching for an available cell provided by the gNB.

At this point:

- The AMF is listening for NGAP connections.
- The gNB is ready to accept UE radio connections.
- The UE is idle and searching for the network.

2. gNB Connects to the AMF

The gNB initiates an NG Setup Request to the AMF. This message contains information such as the supported tracking areas and PLMNs. The AMF validates the information and replies with an NG Setup Response.

After this exchange, the N2 (NGAP) control-plane connection between the gNB and the AMF is successfully established.

3. UE Searches for the Network

After startup, the UE scans available frequencies, detects the gNB, synchronizes with it, reads the broadcast system information, and prepares to establish a connection.

4. RRC Connection Establishment

Before the UE can exchange NAS messages, it must establish an RRC connection with the gNB.

The sequence is:

- RRC Connection Request
- RRC Connection Setup
- RRC Setup Complete

Once these messages are exchanged, a dedicated signaling connection exists between the UE and the gNB.

5. UE Registration (NAS)

The UE sends a Registration Request (NAS message). The gNB does not process this NAS message; instead, it encapsulates the NAS message inside an NGAP message and forwards it to the AMF over the N2 interface.

6. Authentication and Registration

The AMF performs the core network procedures, including:

- UE authentication
- Security establishment

- Identity verification
- Registration processing

If successful, the AMF sends a Registration Accept message back to the UE through the gNB.

7. Connection Complete

After successful registration:

- The UE enters the Registered state.
- The AMF stores the UE context.
- The gNB maintains the radio connection.

The network is now ready for PDU Session Establishment, IP address allocation, and user-plane data transfer.

8. Observing the Procedure in Wireshark

When capturing traffic on the loopback interface and applying the 'ngap' display filter, the primary signaling messages observed include:

- NG Setup Request
- NG Setup Response
- Initial UE Message (containing the Registration Request)
- Downlink NAS Transport
- Uplink NAS Transport
- Initial Context Setup Request
- Initial Context Setup Response
- Registration Accept

These messages represent the signaling exchanged between the gNB and the AMF during UE registration.

Workflow Summary

1. Start Open5GS AMF.
2. Start the Duranta gNB.
3. Perform NG Setup Request/Response.
4. Start the Duranta UE.
5. The UE discovers the gNB and establishes an RRC connection.
6. The UE sends a Registration Request.
7. The gNB forwards the NAS message to the AMF using NGAP.
8. The AMF authenticates and registers the UE.
9. The Registration Accept message is returned through the gNB.
10. The UE is successfully registered and the signaling can be observed in Wireshark.

CLONING THE REPO –

```
~ cd
~ mkdir duranta
~ cd duranta
~ git clone https://github.com/duranta-project/openairinterface5g
```

PHASE 1: BUILDING THE RAN BINARIES –

STEP 1: Run OpenAirInterface's built-in tool to download core tools like cmake, cryptographic libraries, and the asn1c protocol compiler:

```
~ cd ~/duranta/openairinterface5g/cmake_targets
~ ./build_oai -I
```

Note: During this installation, Ubuntu will ask you two interactive questions. Select No for running iperf3 as a background daemon (to save RAM) and Yes for Wireshark non-root packet capture.

STEP 2: Install Missing Optional Libraries Manually install graphics and memory offloading packages to unblock the compiler's optional module scans.

```
~ sudo apt update && sudo apt install -y libgl1-mesa-dev libglu1-mesa-dev libulfius-dev libdpdk-dev
```

STEP 3: Compile the gNB (Cell Tower Binary) Compile the base station, omitting the --build-lib all flag to bypass unnecessary/buggy optional modules. The -w SIMU flag swaps physical hardware antenna drivers for the software-defined loopback simulator.

```
~ ./build_oai --gNB -w SIMU
```

STEP 4: Step 4: Compile the nrUE (Virtual Handset Binary) Compile the virtual phone stack client:

```
~ ./build_oai --nrUE -w SIMU
```

STEP 5: Step 5: Verify Your Executables Ensure the binaries generated successfully:

```
~ ls -la ran_build/build/nr-softmodem ran_build/build/nr-uesoftmodem
```

```
test@LAPTOP-931H1VKT:~/duranta/openairinterface5g/cmake_targets$ ls -la ran_build/build/nr-softmodem ran_build/build/nr-uesoftmodem
-rwxr-xr-x 1 test test 109798224 Jul  2 07:43 ran_build/build/nr-softmodem
-rwxr-xr-x 1 test test  47730072 Jul  2 07:45 ran_build/build/nr-uesoftmodem
```

PHASE 2: CREATING CELL PROFILE CONFIGURATION -

Next, you must give your blank gNB engine an identity and radio grid profile so it knows how to broadcast and where to find the Core Network.

STEP 1: Set Up Workspace and Copy the Profile Template.

```
~ cd ~/duranta/openairinterface5g
~ mkdir ~/my_sims_labs
~ cp ~/duranta/openairinterface5g/targets/PROJECTS/GENERIC-NR-5GC/CONF/gnb.sa.band78.fr1.106PRB.pci0.rfsim.conf
~/duranta/openairinterface5g/my_sim_labs/gnb_light_20mhz.conf
```

PHASE 3: INSTALLING OPEN-5GS CORE AND REGISTERING THE SIM

We use the lightweight, C-based Open5GS backend rather than heavy Docker containers to manage the Core Network.

STEP 1: Add the Repo and Install Open-5GS

```
~ cd
~ sudo add-apt-repository ppa:open5gs/latest -y
~ sudo apt update
~ sudo apt install open5gs -y
```

Verify the core network is installed correctly by running `~ systemctl list-unit-files | grep open5gs`

```
test@LAPTOP-931H1VKT:~$ systemctl list-unit-files | grep open5gs
open5gs-amfd.service          enabled          enabled
open5gs-ausfd.service         enabled          enabled
open5gs-bsfd.service          enabled          enabled
open5gs-hssd.service          enabled          enabled
open5gs-mmed.service          enabled          enabled
open5gs-nrfd.service          enabled          enabled
open5gs-nssfd.service         enabled          enabled
open5gs-pcfd.service          enabled          enabled
open5gs-pcrfd.service         enabled          enabled
open5gs-scpd.service          enabled          enabled
open5gs-seppd.service         enabled          enabled
open5gs-sgwcd.service         enabled          enabled
open5gs-sgwud.service         enabled          enabled
open5gs-smfd.service          enabled          enabled
open5gs-udmd.service          enabled          enabled
open5gs-udrd.service          enabled          enabled
open5gs-upfd.service          enabled          enabled
```

Verify if the network is running by running `~ sudo systemctl status open5gs-amfd`

```
test@LAPTOP-931H1VKT:~$ sudo systemctl status open5gs-amfd
● open5gs-amfd.service - Open5GS AMF Daemon
   Loaded: loaded (/usr/lib/systemd/system/open5gs-amfd.service; en>
   Active: active (running) since Fri 2026-07-03 04:35:56 UTC; 8s a>
   Main PID: 363 (open5gs-amfd)
     Tasks: 2 (limit: 7108)
    Memory: 15.5M ()
   CGroup: /system.slice/open5gs-amfd.service
           └─363 /usr/bin/open5gs-amfd -c /etc/open5gs/amf.yaml
```

You should see Active: Activating or Active: Running.

If you see Active: failed, then run

`~ sudo systemctl start open5gs-amfd`

and check again with `~ systemctl status open5gs-amfd`

STEP 2: Now we will be installing UI version to enter UE sim details into the database.

First we will install the mongodb database services.

Run:

`~ mongod --version`

`~ mongosh --version`

if they are already present no need to install it, if not follow the instructions -

```
~ curl -fsSL https://pgp.mongodb.com/server-8.0.asc | \  
  sudo gpg -o /usr/share/keyrings/mongodb-server-8.0.gpg \  
  --dearmor
```

```
~ echo "deb [ signed-by=/usr/share/keyrings/mongodb-server-8.0.gpg ] \  
https://repo.mongodb.org/apt/ubuntu noble/mongodb-org/8.0 multiverse" | \  
  sudo tee /etc/apt/sources.list.d/mongodb-org-8.0.list
```

```
~ sudo apt update
```

```
~ sudo apt install -y mongodb-org
```

Verify mongo is running by `~ sudo systemctl status mongod`

You should see active: active or active: running. if not then run `~ sudo systemctl start mongod`

And check again `~ sudo systemctl status mongod`

Enable mongo to run on boot by `~ sudo systemctl enable mongod`

STEP 3: Open 5gs UI service is a node process and requires node.js on your machine. We will install node js locally. If you already have it, skip this step.

```
~ sudo apt update  
~ sudo apt install -y nodejs npm
```

Verify using `~ node -v npm -v`.

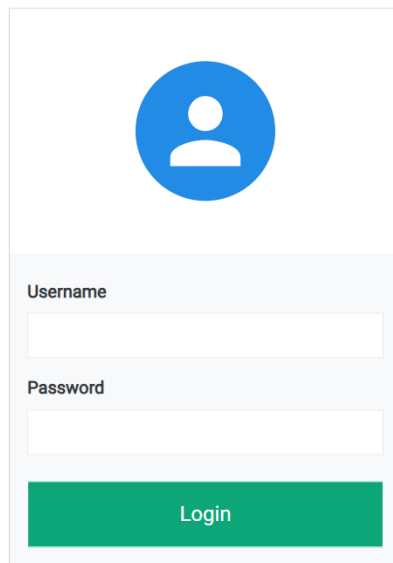
STEP 4: Installing 5gs UI service -

```
~ cd ~  
~ git clone https://github.com/open5gs/open5gs.git  
~ cd ~/open5gs/webui  
~ npm ci
```

```
~ npm run dev
```

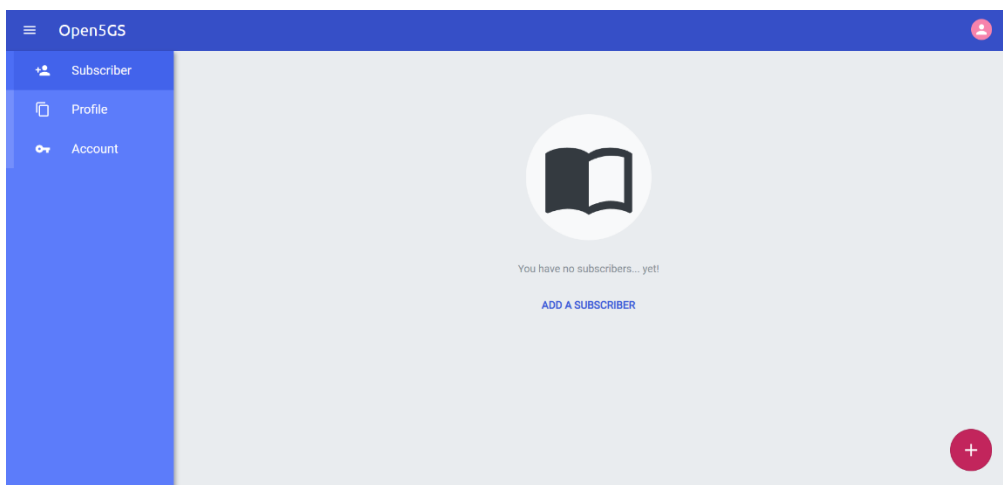
This will start a node js process on you machine at port 9999.

Open your browser and go to `http://localhost:9999`



A login form with a blue circular icon containing a white person silhouette at the top. Below the icon are two input fields: the first is labeled "Username" and the second is labeled "Password". At the bottom of the form is a green rectangular button with the text "Login" in white.

Username: admin
Password: 1423



This is where we will add the details of the UE Sim. We will come back at this later. First we will configure config files for both UE and gNB.

PHASE 4: EDITING THE CONFIGS FILE –

Most Important phase, for the UE, gNB and AMF to talk to each other, the parameters inside the config files for all three must match so they can locate position of each other.

STEP 1: Open three terminals side by side and open config files for each UE, gNB and AMF.

In the first terminal open the UE config file. Run this –

```
~ cd ~/duranta/openairinterface5g/targets/PROJECTS/GENERIC-NR-5GC/CONF  
~ gvim ue.conf
```

If gvim is not installed, run

```
~ sudo apt install vim-gtk3
```

Then run

```
~ gvim ue.conf
```

This will open the UE config file. Keep it open and go to second terminal.

In the second terminal we will open the base station gNB config file. Run these –

```
~ cd ~/duranta/openairinterface5g/my_sim_labs  
~gvim gnb_light_20mhz.conf
```

Now move to third terminal to open the amf config file.

```
~ cd /etc/open5gs/  
~ gvim amf.yaml
```

Now we have all three config files opened and ready for inspection.

STEP 2: Editing the config files:

Look into the gNB file:

```

gNBs =
(
{
////////// Identification parameters:
gNB_ID    = 0xe00;
gNB_name  = "gNB-0AI";

// Tracking area code, 0x0000 and 0xfffe are reserved values
tracking_area_code = 1;
plmn_list = ({ mcc = 001; mnc = 01; mnc_length = 2; snssaiList = ({ sst = 1; }) });
@include "neighbour-config-rfsim.conf"

nr_cellid = 12345678L;

```

The plmn list must match with other config files. Now look into the amf file.

```

amf:
  sbi:
    server:
      - address: 127.0.0.5
        port: 7777
    client:
#     nrf:
#       - uri: http://127.0.0.10:7777
    scp:
      - uri: http://127.0.0.200:7777
  ngap:
    server:
      - address: 127.0.0.5
  metrics:
    server:
      - address: 127.0.0.5
        port: 9090
  guami:
    - plmn_id:
        mcc: 999
        mnc: 70
      amf_id:
        region: 2
        set: 1
  tai:
    - plmn_id:
        mcc: 999
        mnc: 70
      tac: 1
  plmn_support:
    - plmn_id:
        mcc: 999
        mnc: 70
      s_nssai:
        - sst: 1
  security:

```

The mcc and mnc for amf file are 999 and 70 but for gnb file its 001 and 01, which is a conflict. We will change in the gNB file and set:

mcc = 999

mnc = 70.

Open the gNB file. Take the cursor to plmn_list line.

Press I. The file will go in Insert mode.

Carefully set the values mcc = 999 and mnc = 70.

Press esc. This will take you out of the Insert mode.

Also put a hash # before the line

@include "neighbour-config-rfsim.conf"

to comment it out.

Or you can simply remove that line.

Now scroll down in the gNB file and find the SCTP section.

```
# ----- SCTP definitions
SCTP :
{
    # Number of streams to use in input/output
    SCTP_INSTREAMS = 2;
    SCTP_OUTSTREAMS = 2;
};

////////// AMF parameters:
amf_ip_address = ({ ipv4 = "192.168.70.132"; });

NETWORK_INTERFACES :
{
    GNB_IPV4_ADDRESS_FOR_NG_AMF = "192.168.70.129/24";
    GNB_IPV4_ADDRESS_FOR_NGU = "192.168.70.129/24";
    GNB_PORT_FOR_S1U = 2152; # Spec 2152
};
}
```

Look for the amf ip address.

Now go the amf config file.

The address in the amf file is 127.0.0.5. This is a conflict.

Change the amf_ip_address in the gNB file to “127.0.0.5”

Also change the GNB_IPV4_ADDRESS_FOR_NG_AMF and GNB_IPV4_ADDRESS_FOR_NGU both to “127.0.0.1/24”

Point to cursor to the desired locations.

Press I.

Make changes.

Press esc.

```
amf:
  sbi:
    server:
      - address: 127.0.0.5
        port: 7777
    client:
#     nrf:
#       - uri: http://127.0.0.10:7777
    scp:
      - uri: http://127.0.0.200:7777
  ngap:
    server:
      - address: 127.0.0.5
  metrics:
    server:
      - address: 127.0.0.5
        port: 9090
```

Now in the UE config file, change the IMSI value.

Set IMSI value – “999700000000001”

Press i. Edit the value. Press esc.

Now comes the saving and closing part. Repeat the same process with each file to save the changes and exit for gvim editor.

Press :

Write wq

Press enter.

This will save the changes. Open the files again to make sure the changes are made correctly.

PHASE 5: Adding the subscriber in the database.

This is done for authentication between UE and AMF.

Open the UE conf file first. The same parameters will be inserted in the databse.

```
~ cd ~/duranta/openairinterface5g/targets/PROJECTS/GENERIC-NR-5GC/CONF
~ gvim ue.conf
```

Now open the Open 5gs UI application to insert the subscriber.

```
~ cd ~/open5gs/webui
~ npm run dev
```

Open <http://localhost:9999> on your browser.

Login using:

```
{
    Username: admin,
    Password: 1423
}
```

The web ui will open. Click on add subscriber.

Copy the fields from the ue.conf file to the Create Subscriber.

Enter –

1. Imsi
2. Key
3. Opc
4. Sst
5. DNN

The screenshot shows the 'Create Subscriber' web application interface. The title bar at the top says 'Create Subscriber'. Below it, the section is titled 'Subscriber Configuration'. There is a red-bordered input field for 'IMSI*' with a red asterisk and a red error message 'is required' below it. To the right of this field is a blue button with a white plus sign. Below the IMSI field, there are two input fields: 'Subscriber Key (K)*' with the value '465B5CE8 B199B49F AA5F0A2E E238A6BC' and 'Authentication Management Field (AMF)*' with the value '8000'. Below these, there is a dropdown menu for 'USIM Type' with 'OPc' selected, and an input field for 'Operator Key (OPc/OP)*' with the value 'E8ED289D EBA952E4 283B54E8 8E6183CA'. At the bottom, there are four input fields: 'UE-AMBR Downlink*' with '1', 'Unit' with 'Gbps', 'UE-AMBR Uplink*' with '1', and 'Unit' with 'Gbps'. At the bottom right, there are two buttons: 'CANCEL' and 'SAVE'.

Edit Subscriber

Operator Determined Barring (TS 29.272 7.3.30)

(0) All Packet Oriented Services Barred

Slice Configurations

SST*

☒ 1
☐ 2
☐ 3
☐ 4

SD

☒ Default S-NSSAI

Session Configurations

DNN/APN*

oai

Type*

IPv4v6

☐ LBO Roaming Allowed

5QI/QCI*

9

ARP Priority Level (1-15)*

CANCEL

SAVE

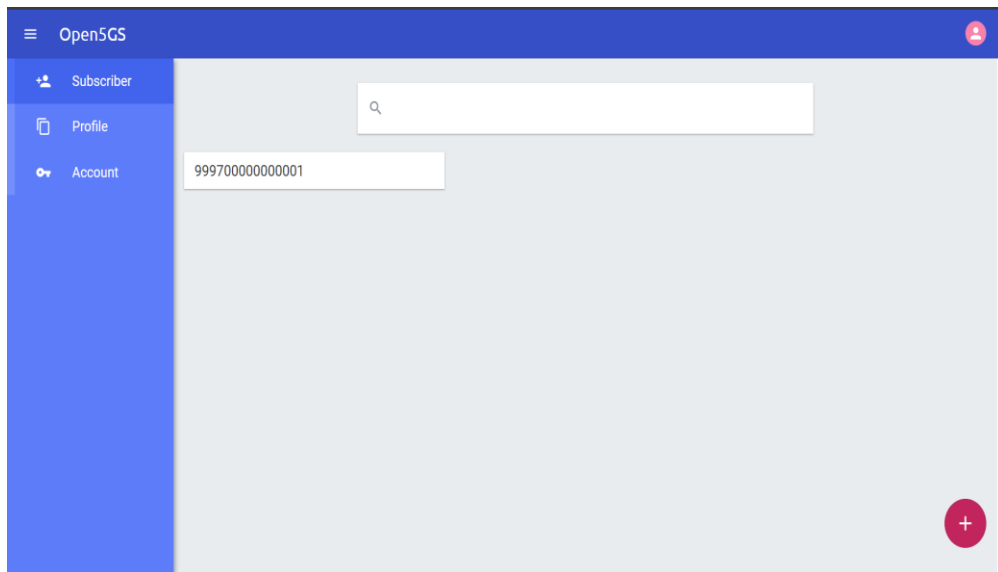
Don't change any other fields. Press save and you will the subscriber got created.

```
# SPDX-License-Identifier: LicenseRef-CSSL-1.0

uicc0 = {
imsi = "999700000000001";
key = "fec86ba6eb707ed08905757b1bb44b8f";
opc= "C42449363BBAD02B66D16BC975D77CC1";
pdu_sessions = ({ dnn = "oai"; nssai_sst = 1; });
}

position0 = {
    x = 0.0;
    y = 0.0;
    z = 6377900.0;
}

@include "channelmod_rfsimu_LE0_satellite.conf"
```



PHASE 6: RUNNING THE BINARIES –

Step 1: Check if the open 5gs network is running

~ sudo systemctl status open5gs-amfd

~ sudo systemctl status open5gs-smfd

~ sudo systemctl status open5gs-upfd

All must be Active: active (running). If not then run –

~ sudo systemctl start open5gs-{process_name}

~ sudo ss -ltnpS | grep open5gs-amfd

```
test@LAPTOP-931H1VKT:~/duranta/openairinterface5g/targets/PROJECTS/GENERIC-NR-5GC/CONF$ sudo ss -ltnpS | grep open5gs-amfd
[sudo] password for test:
tcp  LISTEN  0      4096      127.0.0.5:9090      0.0.0.0:*    users:(("open5gs-amfd",pid=441483,fd=9))
tcp  LISTEN  0      5        127.0.0.5:7777      0.0.0.0:*    users:(("open5gs-amfd",pid=441483,fd=10))
sctp LISTEN  0      5        127.0.0.5:38412     0.0.0.0:*    users:(("open5gs-amfd",pid=441483,fd=11))
test@LAPTOP-931H1VKT:~/duranta/openairinterface5g/targets/PROJECTS/GENERIC-NR-5GC/CONF$ |
```

AMF must be listening on all these 3 ports.

STEP 2: Starting the gNB Base Station.

```
~ cd ~/duranta/openairinterface5g/cmake_targets/ran_build/build
```

```
~ sudo ./nr-softmodem -O ~/duranta/openairinterface5g/my_sim_labs/gnb_light_20mhz.conf  
--rfsim --gNBs.[0].min_rxtxtime 3
```

This will start the gNB and you will start seeing logs. Look for the specific logs ->

```
[GTPU] Created gtpu instance id: 80  
[UTIL] threadCreate() for GTPrx_80: creating thread with affinity ffffffff, priority 50  
[NGAP] Send NGSetupRequest to AMF  
[NGAP] 3584 -> 0000e000  
[UTIL] threadCreate() for TASK_GNB_APP: creating thread with affinity ffffffff, priority 50  
[NR_RRC] Accepting new CU-UP ID 3584 name gNB-OAI (assoc_id -1)  
[UTIL] threadCreate() for TASK_GTPV1_U: creating thread with affinity ffffffff, priority 50  
[UTIL] threadCreate() for time source iq samples: creating thread with affinity fffffff, priority 2  
[UTIL] time manager configuration: [time source: iq_samples] [mode: standalone] [server IP: 127.0.0.1] [server port: 7374] (server IP/port not used)  
[NGAP] Served GUAMIs for AMF open5gs-amf0 (assoc_id=25):  
[NGAP] GUAMI:  
[NGAP] PLMN: MCC=999, MNC=70  
[NGAP] AMF Region ID: 2  
[NGAP] AMF Set ID: 1  
[NGAP] AMF Pointer: 0  
[NGAP] Supported PLMN 0: MCC=999 MNC=70  
[NGAP] Supported slice (PLMN 0): SST=0x01 SD=000  
[ITTI] tried to remove a non existing timer  
[NGAP] reconnected to AMF  
[NGAP] Received NGSetupResponse from AMF  
[GNB_APP] [gNB 0] Received NGAP_REGISTER_GNB_CNF: associated AMF 1  
[PHY] RU clock source set as internal
```

AMF and gNB handshake completes with our given config parameters.

```
[PHY] RU 0 RF started cpu_meas_enabled 0  
[HW] No connected device, generating void samples...  
[PHY] Command line parameters for OAI UE: -C 3619200000 -r 106 --numerology 1 --ssb 516  
[NR_MAC] Frame.Slot 128.0  
  
[NR_MAC] Frame.Slot 256.0  
  
[NR_MAC] Frame.Slot 384.0  
  
[NR_MAC] Frame.Slot 512.0  
  
[NR_MAC] Frame.Slot 640.0  
  
[NR_MAC] Frame.Slot 768.0  
  
[NR_MAC] Frame.Slot 896.0  
  
[NR_MAC] Frame.Slot 0.0  
  
[NR_MAC] Frame.Slot 128.0  
  
[NR_MAC] Frame.Slot 256.0
```

This means the station is listening but since we have not ran the UE, its showing void samples.

Now we will run the UE binary. Keep the base station running and open another terminal

STEP 3: Running the UE binary.

```
~ cd ~/duranta/openairinterface5g/cmake_targets/ran_build/build
```

```
~ sudo ./nr-uesoftmodem -O ~/duranta/openairinterface5g/targets/PROJECTS/GENERIC-NR-5GC/CONF/ue.conf -C 3619200000 --band 78 --ssb 516 -r 106 -rfsim
```

This will start the UE and it will connect to the gNB. Look for logs in both terminals.

UE logs -

```
[PHY] [UE thread Synch] Running Initial Synch
[NR_PHY] Starting cell search with center freq: 3619200000, bandwidth: 106. Scanning for 1 number of GSCN.
[NR_PHY] Scanning GSCN: 0, with SSB offset: 516, SSB Freq: 0.000000
[PHY] Initial sync: pbch decoded successfully, ssb index 0
[PHY] pbch rx ok. rsrp:51 dB/RE, adjust_rxgain:-1 dB
[NR_PHY] Cell Detected with GSCN: 0, SSB SC offset: 516, SSB Ref: 0.000000, PSS Corr peak: 0 dB, PSS Corr Average: 0
[PHY] [UE0] In synch, rx_offset 184320 samples
[PHY] [UE 0] Measured Carrier Frequency offset 0 Hz
[PHY] Initial sync successful, PCI: 0
[PHY] HW: Configuring channel 0 (rf_chain 0): setting tx_freq 3619200000 Hz, rx_freq 3619200000 Hz, tune_offset 0
[PHY] Got synch: hw_slot_offset 12, carrier off 0 Hz
[NR_RRC] [UE 0] BCCH update: phyCellID 0->0, arfcn_ssb 0->641280
[PHY] UE synchronized! decoded_frame_rx=412 UE->init_sync_frame=0 trashed_frames=38
[PHY] Resynchronizing RX by 184320 samples
[HW] received write reorder clear context
[PHY] max_pos_acc = 0, shiftForNextFrame = 0
[PHY] Frame 452, slot 10, SSB Index 0. Error decoding PBCH!
[NR_RRC] SIB1 decoded
[NR_MAC] [UE 0] Initial cell selection: dl_frequency=3600120 kHz (from command-line, band=78, scs=1)
[PHY] PRACH [UE 0] in frame.slot 453.19, placing PRACH in position 2828, Msg1/MsgA-Preamble frequency start 0 (kl 0, mbleIndex = 24)
[PHY] [UE 0] RAR-Msg2 decoded
[NR_MAC] [UE 0][RAPROC][RA-RNTI 010f] Got BI RAR subPDU 5 ms
[NR_MAC] [UE 0][RAPROC][RA-RNTI 010f] Got RAPID RAR subPDU
[NR_MAC] [UE 0][RAPROC][454.12] Found RAR with the intended RAPID 24
[MAC] received TA command 31
[NR_MAC] [RAPROC][454.18] RA-Msg3 transmitted
[NR_MAC] [UE 0][456.10][RAPROC] 4-Step RA procedure succeeded. CBRA: Contention Resolution is successful.
[NR_RRC] [UE0][RAPROC] Logical Channel DL-CCCH (SRB0), Received NR_RRCSetup
[RLC] Added srb 1 to UE 0
[NR_RRC] State = NR_RRC_CONNECTED
[NAS] Generate Initial NAS Message: Registration Request
[NAS] [UE 0] Received NR_NAS_CONN_ESTABLISH IND: asCause 0
[PHY] UL hard: 5/70 avg code rate 0.1, avg bit/symbol 2.1, avg per TB: (nb RBS 11.3, nb symbols 10.2)
[SDAP] Could not find SDAP entity: entity list empty (ue_id=0 pdu_session_id=1)
[SDAP] Creating SDAP entity ue_id=0 pdu_session_id=1
[SDAP] Default DRB for the created SDAP entity: DRB 1
[PDCCP] Added DRB 1 to UE ID 0
[NR_RRC] State = NR_RRC_CONNECTED
[NR_RRC] RRCReconfiguration includes Measurement Configuration
[NR_RRC] Measurement gaps not yet supported!
[NAS] [UE 0] Received NAS_CONN_ESTABLISH CNF: errCode 1, length 63
[NR_RRC] rrcReconfigurationComplete Encoded 10 bits (2 bytes)
[NR_RRC] Logical Channel UL-DCCH (SRB1), Generating RRCReconfigurationComplete (bytes 2)
[NAS] Received PDU Session Establishment Accept, UE IPv4: 10.45.0.3
[SDAP] UE 0 PDU session 1: cached QFI 1
[SDAP] UE 0 PDU session 1: bringing TUN oaitun_ue1 up
[UTIL] threadCreate() for ue_tun_read_0_p1: creating thread with affinity ffffffff, priority 1
[OIP] TUN Interface oaitun_ue1 successfully configured, IPv4 10.45.0.3, IPv6 (null)
[NR_PHY] RI = 1 i1 = 0.0.0, i2 = 0, SINR = 43 dB, CQI = 15
```

gNB logs –

```
[NR_MAC] Frame.Slot 256.0

[HW] Client connects from ::ffff:127.0.0.1:2268
[HW] RFSim: Number of antennas changed from 0 to 1
[NR_MAC] Frame.Slot 384.0

[NR_PHY] [RAPPROC] 453.19 Initiating RA procedure with preamble 24, energy 43.3 dB (I0 0, thres 120), delay 0 start symbol 4 freq index 0
[NR_MAC] 453.19 UE RA-RNTI 010f TC-RNTI f9cf: initiating RA procedure
[NR_MAC] UE f9cf: Sync beam index 0
[NR_MAC] UE f9cf: Msg3 scheduled at 454.18 (454.12 TDA 0) start 0 RBs 8
[NR_MAC] UE f9cf: 454.12 Generating RA-Msg2 DCI, RA RNTI 0x10f, state 1, preamble_index(RAPID) 24, timing_offset = 0 (estimated distance 0.0 [m])
[NR_MAC] 454.18 PUSCH with TC-RNTI 0xf9cf received correctly
[MAC] [RAPPROC] Received SDU for CCCH length 6 for UE f9cf
[RLC] Activated srb0 for UE 63951
[NR_RRC] SRS configured with 1 ports
[RLC] Added srb 1 to UE 63951
[NR_MAC] Activating scheduling Msg4 for TC-RNTI 0xf9cf (state WAIT_Msg3)
[NR_RRC] Decoding CCCH: RNTI f9cf, payload_size 6
[NR_RRC] [---] (cellID 0, UE ID 1 RNTI f9cf) Create UE context: CU UE ID 1 DU UE ID 63951 (rnti: f9cf, random ue id 9eea849ab3000000)
[NR_RRC] Adding new PCell (cell 12345678, assoc_id -1)
[RRC] activate SRB 1 of UE 1
[NR_RRC] [DL] (cellID bc614e, UE ID 1 RNTI f9cf) Send RRC Setup
[NR_MAC] UE f9cf Generate Msg4: feedback at 456.17, payload 225 bytes, next state nrRA_WAIT_Msg4_MsgB_ACK
[NR_MAC] 456.17 UE f9cf: Received Ack of Msg4. CBRA procedure succeeded (UE Connected)
[NR_MAC] Adding new UE context with RNTI 0xf9cf
[NR_RRC] [UL] (cellID bc614e, UE ID 1 RNTI f9cf) Received RRCSetupComplete (RRC_CONNECTED reached)
[NGAP] Selected PLMN in the NG Initial UE Message: MCC=999 MNC=70
[NGAP] UE 1: Selected AMF 'open5gs-amf0' (assoc_id 29) through selected PLMN MCC=999 MNC=70
[NGAP] Create UE context (ID 1) for AMF 'open5gs-amf0' (assoc_id 29)
```

```
[NGAP] Create UE context (ID 1) for AMF 'open5gs-amf0' (assoc_id 29)
[NR_MAC] invalid cgi_idx 0, default to MCS 9
[NR_RRC] [DL] (cellID bc614e, UE ID 1 RNTI f9cf) Send DL Information Transfer [42 bytes]
[NR_RRC] [UL] (cellID bc614e, UE ID 1 RNTI f9cf) Received RRC UL Information Transfer [24 bytes]
[NR_MAC] Frame.Slot 512.0
UE RNTI f9cf CU-UE-ID 1 in-sync PH 48 dB PCMAX 20 dBm, average RSRP -43 (6 meas)
UE f9cf: CQI 15, RI 1, PMI (0,0)
UE f9cf: UL-RI 1, TPMI 0
UE f9cf: dl_sch_rounds 4/0/0/0, dl_sch_errors 0, pucch0_DTX 0 (SNR 38.3+18.3 dB), BLER 0.09000 MCS (0) 0 CCE fail 0, goodput 0.00 Mbps
UE f9cf: ulsch_rounds 10/0/0/0, ulsch_errors 0, ulsch_DTX 0, BLER 0.06561 MCS (0) 0 (Qm 2 deltaMCS 0 dB) NPRB 5 SNR 27.2 (+12.2) dB CCE fail 0, goodput 0.00 Mbps
UE f9cf: LCID 1: TX 59 RX 80 bytes

[NR_RRC] [DL] (cellID bc614e, UE ID 1 RNTI f9cf) Send DL Information Transfer [19 bytes]
[NR_RRC] [UL] (cellID bc614e, UE ID 1 RNTI f9cf) Received RRC UL Information Transfer [60 bytes]
[NR_MAC] Frame.Slot 640.0
UE RNTI f9cf CU-UE-ID 1 in-sync PH 48 dB PCMAX 20 dBm, average RSRP -43 (16 meas)
UE f9cf: CQI 15, RI 1, PMI (0,0)
UE f9cf: UL-RI 1, TPMI 0
UE f9cf: dl_sch_rounds 7/0/0/0, dl_sch_errors 0, pucch0_DTX 0 (SNR 35.3+15.3 dB), BLER 0.07290 MCS (0) 0 CCE fail 1, goodput 0.00 Mbps
UE f9cf: ulsch_rounds 27/0/0/0, ulsch_errors 0, ulsch_DTX 0, BLER 0.01660 MCS (0) 0 (Qm 2 deltaMCS 0 dB) NPRB 5 SNR 21.8 (+6.8) dB CCE fail 0, goodput 0.00 Mbps
UE f9cf: LCID 1: TX 92 RX 155 bytes

[NGAP] NGAP_FIND_PROTOCOLIE_BY_ID is NULL (searching for ie: 110)
[NGAP] NGAP_FIND_PROTOCOLIE_BY_ID is NULL (searching for ie: 71)
[NGAP] AllowedNSSAIList count 1
[NGAP] NGAP_FIND_PROTOCOLIE_BY_ID is NULL (searching for ie: 36)
[NGAP] Initial Context Setup UE RAN ID 1 UE AMF ID 1: 0 PDU session(s)
[NR_RRC] [---] (cellID bc614e, UE ID 1 RNTI f9cf) Selected security algorithms: ciphering 0, integrity 2
[NR_RRC] [UE f9cf] Saved security key 95
[NR_RRC] UE 1 Logical Channel DL-DCCH, Generate SecurityModeCommand (bytes 3)
[NR_RRC] [UL] (cellID bc614e, UE ID 1 RNTI f9cf) Received Security Mode Complete
[NR_RRC] UE 1: Logical Channel DL-DCCH, Generate NR UE Capability Enquiry (bytes 0, xid 1)
[NR_RRC] [UL] (cellID bc614e, UE ID 1 RNTI f9cf) Received UE capabilities
```

```
[NR_RRC] [UL] (cellID bc614e, UE ID 1 RNTI f9cf) Received UE capabilities
[NR_RRC] Send message to ngap: NGAP_UE_CAPABILITIES_IND
[NR_RRC] [DL] (cellID bc614e, UE ID 1 RNTI f9cf) Send DL Information Transfer [46 bytes]
[NR_RRC] Send message to sctp: NGAP_InitialContextSetupResponse
[NR_RRC] [UL] (cellID bc614e, UE ID 1 RNTI f9cf) Received RRC UL Information Transfer [13 bytes]
[NR_RRC] [UL] (cellID bc614e, UE ID 1 RNTI f9cf) Received RRC UL Information Transfer [35 bytes]
[NR_RRC] [DL] (cellID bc614e, UE ID 1 RNTI f9cf) Send DL Information Transfer [51 bytes]
[NGAP] PDUSESSIONSetup initiating message
[NR_RRC] UE 1: received PDU Session Resource Setup Request
[NR_RRC] UE 1: added QoS flow with QFI=1, total number of QoS flows = 1
[NR_RRC] Added PDU Session 1, (total nb of sessions = 1)
[NR_RRC] UE 1: created new DRB 1 for QFI 1 (5QI 9)
[NR_RRC] UE 1: assigned DRB 1 to QFI 1 (5QI 9) in PDU session 1
[NR_RRC] PDU Session to Setup: PDU Session ID=1, incoming TEID=0x0000530c, Addr=127.0.0.7
[NR_RRC] [---] (cellID bc614e, UE ID 1 RNTI f9cf) selecting CU-UP ID 3584 based on exact NSSAI match (1:0xffffffff)
[RRC] UE 1 associating to CU-UP assoc_id -1 out of 1 CU-UPs
[E1AP] UE 1: set up PDU session ID 1 (1 bearers)
[SDAP] Could not find SDAP entity: entity list empty (ue_id=1 pdu_session_id=1)
[SDAP] Creating SDAP entity ue_id=1 pdu_session_id=1
[SDAP] Default DRB for the created SDAP entity: DRB 1
[PDCP] Added DRB 1 to UE ID 1
[GTU] N3 GTP-U tunnel: PDUSESSION=1/UL TEID=0x0000530c
[GTU] [81] UE ID 1: Create tunnel TEID incoming 0x8c1fa84b outgoing 0x530c to remote IPv4 127.0.0.7, IPv6 ::, port 2152
[RRC] activate SRB 2 of UE 1
[RLC] Added srb 2 to UE 63951
[RLC] Added drb 1 to UE 63951
[RLC] Added DRB to UE 63951
[NR_RRC] SRS configured with 1 ports
[RRC] UE 1 trigger UE context setup request with 1 DRBs
[E1AP] UE 1: updating PDU session ID 1 (0 setup, 1 modify bearers)
[PDCP] DRB 1 re-established
[NR_RRC] PDU Session New: status to ESTABLISH for PDU Session 1
[NR_RRC] [DL] (cellID bc614e, UE ID 1 RNTI f9cf) Generate RRCReconfiguration (bytes 319, xid 0)
[NR_RRC] Received E1AP Bearer Context Modification Response for UE CU-CP ID 1
[RRC] UE 1: PDU session ID 1 modified 1 bearers, setup 0 bearers
[NR_MAC] Received ACK for RRCReconfiguration, but nothing to apply!
[NR_MAC] Frame.Slot 768.0
```

```

[PDCP] DRB 1 re-established
[NR_RRC] PDU Session New: status to ESTABLISH for PDU Session 1
[NR_RRC] [UL] (cellID bc614e, UE ID 1 RNTI f9cf) Generate RRCReconfiguration (bytes 319, xid 0)
[NR_RRC] Received E1AP Bearer Context Modification Response for UE CU-CP ID 1
[RRC] UE 1: PDU session ID 1 modified 1 bearers, setup 0 bearers
[NR_MAC] Received ACK for RRCReconfiguration, but nothing to apply!
[NR_MAC] Frame.Slot 768.0
UE RNTI f9cf CU-UE-ID 1 in-sync PH 48 dB PCMAX 20 dBm, average RSRP -43 (16 meas)
UE f9cf: CQI 15, RI 1, PMI (0,0)
UE f9cf: UL-RI 1, TPMI 0
UE f9cf: dlsch_rounds 18/0/0/0, dlsch_errors 0, pucch0_DTX 0 (SNR 24.3+4.3 dB), BLER 0.04783 MCS (0) 0 CCE fail 4, goodput 0.01 Mbps
UE f9cf: ulsch_rounds 56/0/0/0, ulsch_errors 0, ulsch_DTX 0, BLER 0.00471 MCS (0) 0 (Qm 2 deltaMCS 0 dB) NPRB 5 SNR 18.5 (+3.5) dB CCE fail 0, goodput 0.00 Mbps
UE f9cf: LCID 1: TX 578 RX 274 bytes
UE f9cf: LCID 2: TX 0 RX 0 bytes
UE f9cf: LCID 4: TX 0 RX 0 bytes

[NR_RRC] [UL] (cellID bc614e, UE ID 1 RNTI f9cf) Received RRCReconfigurationComplete
[NR_RRC] PDU Session Setup: ID=1, outgoing TEID=0x8c1fa84b, Addr=127.0.0.1
[NR_RRC] NGAP_PDUSESSION_SETUP_RESP: sending the message
[NR_MAC] DU received confirmation of successful RRC Reconfiguration
[NGAP] Encoded PDU Session Transfer (1): TEID=0x8c1fa84b, Addr=127.0.0.1
[NR_MAC] Frame.Slot 896.0
UE RNTI f9cf CU-UE-ID 1 in-sync PH 48 dB PCMAX 20 dBm, average RSRP -43 (16 meas)
UE f9cf: CQI 15, RI 1, PMI (0,0)
UE f9cf: UL-RI 1, TPMI 0
UE f9cf: dlsch_rounds 21/0/0/0, dlsch_errors 0, pucch0_DTX 0 (SNR 21.3+1.3 dB), BLER 0.03874 MCS (0) 0 CCE fail 4, goodput 0.00 Mbps
UE f9cf: ulsch_rounds 74/0/0/0, ulsch_errors 0, ulsch_DTX 0, BLER 0.00133 MCS (0) 0 (Qm 2 deltaMCS 0 dB) NPRB 5 SNR 17.6 (+2.6) dB CCE fail 0, goodput 0.00 Mbps
UE f9cf: LCID 1: TX 581 RX 284 bytes
UE f9cf: LCID 2: TX 0 RX 0 bytes
UE f9cf: LCID 4: TX 3 RX 65 bytes

[NR_MAC] Frame.Slot 0.0
UE RNTI f9cf CU-UE-ID 1 in-sync PH 48 dB PCMAX 20 dBm, average RSRP -43 (16 meas)
UE f9cf: CQI 15, RI 1, PMI (0,0)
UE f9cf: UL-RI 1, TPMI 0
UE f9cf: dlsch_rounds 22/0/0/0, dlsch_errors 0, pucch0_DTX 0 (SNR 21.3+1.3 dB), BLER 0.03487 MCS (0) 0 CCE fail 4, goodput 0.00 Mbps
UE f9cf: ulsch_rounds 87/0/0/0, ulsch_errors 0, ulsch_DTX 0, BLER 0.00034 MCS (0) 0 (Qm 2 deltaMCS 0 dB) NPRB 5 SNR 17.8 (+2.8) dB CCE fail 0, goodput 0.00 Mbps
UE f9cf: LCID 1: TX 581 RX 284 bytes
UE f9cf: LCID 2: TX 0 RX 0 bytes
UE f9cf: LCID 4: TX 3 RX 65 bytes

```

BOTH GNB AND UE ARE WORKING AND TALKING TO EACH OTHER.

THE SETUP IS SUCCESSFUL.